

PRANA: Policy for Robotic Action via Neural Architecture

Siddarth Dayasagar Arpit Gandhi

dayasagar.s@northeastern.edu

gandhi.arp@northeastern.edu

Abstract

PRANA is a vision-action policy for low-data manipulation on a SO-ARM101 arm. The main contribution is a pair of architectures, v2 and v3, that we developed, trained, and deployed on physical hardware. Both share an encoder-decoder transformer trained with flow matching; noise at inference is drawn from the empirical action covariance, and chunk boundaries are smoothed by a flow-consistent inpainting scheme. v3 differs from v2 by one choice: the visual backbone is swapped from a frozen ViT-Tiny to a frozen DINOv2 ViT-S/14, giving denser and more spatially grounded patch features. Both models are trained on 123 teleoperation episodes collected through LeRobot and deployed on a single 8 GB consumer GPU. An earlier deterministic baseline (v1) is retained for comparison.

Keywords: imitation learning, flow matching, action chunking, robot manipulation, DINOv2

1 Introduction

Learning manipulation policies from demonstrations is appealing, but most published recipes assume hundreds or thousands of episodes, heavy pretraining, or compute that is not available to a student with a single arm and a laptop GPU. Our working constraint is different: fewer than 150 episodes, one RTX-class laptop GPU, and a physical robot that has to pick something up this afternoon.

PRANA is our attempt to build a serviceable policy under those constraints. The task is a pick-and-place: grasp a screwdriver from a table and drop it into a box, using a SO-ARM101 6-DOF arm [11] with a table-mounted camera and a wrist-mounted camera. Data collection and deployment both go through the LeRobot framework. We went through three iterations:

- **v1.** Action-chunking transformer with a single self-attention stack and L1 regression. Baseline.
- **v2.** Flow-matching encoder-decoder with correlated noise at inference and rolling inpainting. First version that reliably completed the task.
- **v3.** v2 with the ViT-Tiny backbone replaced by a frozen DINOv2 ViT-S/14. Same decoder, same loss, same noise.

The claims here are practical rather than theoretical. Flow matching scales down to ~ 120 episodes if the inference noise is chosen carefully. A separate encoder and decoder prevents the action queries from poisoning the visual context. Fitting the Cholesky factor of the action covariance into the checkpoint after training removes deployment jitter. A self-supervised backbone improves grasp precision without touching the decoder. Everything runs on a single 8 GB laptop GPU through LeRobot.

2 Related Work

2.1 Vision-Language-Action Models

VLA models fine-tune pretrained vision-language backbones for robot control. RT-2 [4] showed that LLM representations transfer to action prediction. Pi0 [2] introduced flow matching as an action head running up to 50 Hz, and Pi0.5 [3] extended this with a separate action expert. PRANA borrows the flow-matching idea but drops the VLM backbone entirely. At our scale, there is no demonstration budget to fine-tune one sensibly.

2.2 Action Chunking

ACT [1] established that predicting a short action sequence rather than a single step improves temporal consistency and makes open-loop execution viable. It handled multimodality with a CVAE. PRANA v1 follows the chunking idea without the CVAE; v2 replaces the CVAE with flow matching, which we found easier to tune in the low-data regime.

2.3 Flow Matching and Diffusion for Actions

Flow matching [6] and diffusion policies [7] both give expressive action distributions that handle multimodal demonstrations better than deterministic regression. The first-place BEHAVIOR 2025 entry [5] uses flow matching with correlated noise sampled from $\mathcal{N}(0, \beta\Sigma + (1 - \beta)I)$, where Σ is the empirical action covariance. They argued this reduces the asymmetry between early and late denoising steps. We adapt the covariance idea in a simpler form, applied only at inference.

2.4 Self-Supervised Vision for Robotics

DINOv2 [10] produces patch-level features with strong spatial semantics via self-distillation on a curated 142M-image dataset. For manipulation, where “where is the shaft” matters more than “is this a screwdriver,” local spatial features tend to help more than class-discriminative ones.

2.5 Temporal Consistency in Chunked Execution

Real-time chunking [9] proposes rolling execution with inpainting to balance temporal smoothness against reactivity. The BEHAVIOR entry extended this with correlation-aware inpainting that propagates corrections using the learned action structure. We use a simpler variant: the first few positions of each new chunk are forced onto the flow interpolant of the previous chunk’s tail, but only while the denoising time is above a threshold.

3 Background: PRANA v1

v1 is the baseline we keep around for reference. It is an ACT-style action-chunking transformer: tokens from two cameras (ViT-Tiny, ImageNet-pretrained, unfrozen) at 197 tokens each, a linear state projection, a randomly-initialized embedding

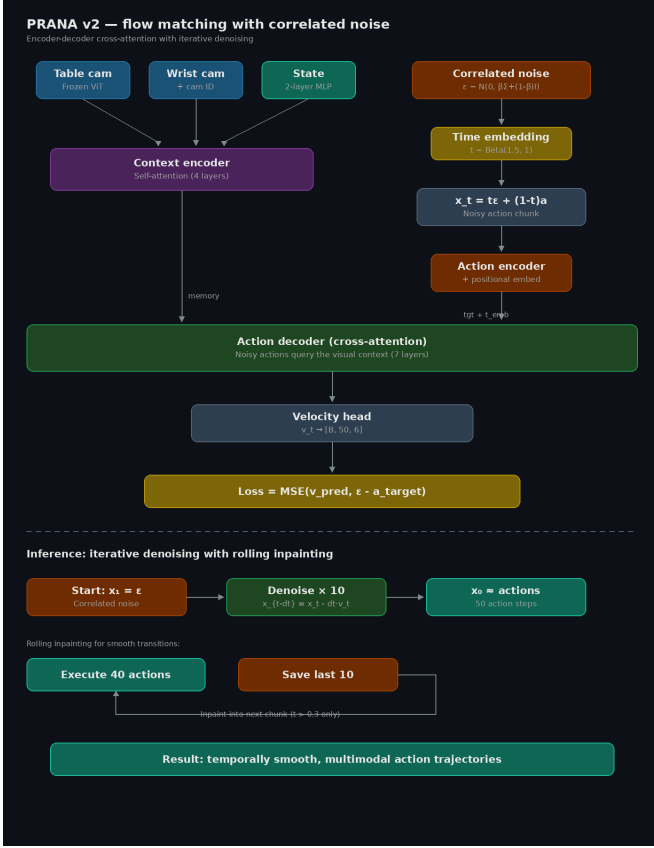


Figure 1: PRANA v2. The context encoder runs self-attention over vision and state tokens. Noisy actions, with positional and time embeddings, are processed by a cross-attention decoder that predicts the velocity field. Inference uses 10 Euler steps, with inpainting on the first 10 positions above a time threshold.

table the size of the PaliGemma vocabulary (a leftover from an early attempt at language conditioning), and 50 learned action queries are concatenated into a single sequence of 461 tokens and processed by a 4-layer TransformerEncoder with self-attention across the whole thing. A 2-layer MLP head regresses the final 50 positions to 6-DOF actions, trained with L1. Two things sink it in practice. Shared self-attention lets the action queries write into the visual context at every layer, and the unfrozen ViT-Tiny overfits inside the first few thousand gradient steps on our dataset. The language stream is never used at deployment — we pass zero token ids through the policy — so roughly 260 MB of the checkpoint is dead weight. v2 reworks all three choices.

4 PRANA v2/v3 Architecture

v2 and v3 share the same decoder, the same flow-matching objective, the same correlated noise sampler, and the same inpainting scheme. They differ only in the visual backbone. We describe the shared architecture first, then cover the two backbones side by side. Figure 1 shows v2 in full; Figure 2 shows the v3 variant.

4.1 Overview

The model has four components. A context encoder (4 layers, 8 heads, FFN 512, pre-norm) builds a scene representation from vision and state tokens via self-attention. An action decoder

(7 layers, same head count and FFN width) cross-attends from noisy action tokens to the encoder output without writing back into the context. A flow-matching objective trains the decoder to predict a velocity field from noise to actions. A correlated noise sampler — fit post-hoc from the training data — provides the initial distribution at inference. With the backbone frozen, trainable parameters total about 8M in both versions.

4.2 Vision Encoder with Camera Identity

Both versions freeze the backbone for the entire training run. This is a direct response to v1: with 123 episodes, an unfrozen ViT-Tiny overfits almost immediately, and we had no practical way to stop it short of a grid search we could not afford. Freezing keeps the pretrained structure intact and pushes adaptation onto the downstream modules.

Each camera’s tokens receive a learned camera-identity embedding added elementwise at the backbone’s native dimension, *before* projection to the 256-dim model space. This gives downstream layers an unambiguous signal for which viewpoint each token came from, which positional encoding alone does not provide. The specifics differ between the two versions:

- **v2:** **timm ViT-Tiny** [8] at 224×224 , patch size 16, ImageNet-pretrained, with the classification head removed and no global pool. It returns all 197 tokens (196 patches + CLS). Camera embedding $e_{\text{cam}} \in \mathbb{R}^{192}$. A linear layer maps $192 \rightarrow 256$ followed by LayerNorm. Two cameras produce $197 \times 2 = 394$ context tokens.
- **v3:** **DINOv2 ViT-S/14** [10] at 224×224 , patch size 14, loaded via `torch.hub`. We read `x_norm_patchtokens` from `forward_features`, which gives 256 patch tokens at 384 dim with CLS dropped. Camera embedding $e_{\text{cam}} \in \mathbb{R}^{384}$. A linear layer maps $384 \rightarrow 256$ followed by LayerNorm. Two cameras produce $256 \times 2 = 512$ context tokens.

The DINOv2 swap gives roughly twice the per-patch dimensionality, about 30% more tokens per camera, a finer receptive field (14 vs. 16 pixels), and self-supervised rather than class-discriminative features. Section 5 returns to what this buys us.

4.3 State Encoder

A 2-layer MLP with GELU and LayerNorm maps the 6-DOF joint state to a single 256-dim token:

$$s_{\text{token}} = \text{LN}(W_2 \text{GELU}(W_1 q_{\text{joint}})). \quad (1)$$

v1 used a single linear layer here, and we found it under-expressive for the velocity-like structure in the joint state.

4.4 Context Encoder

Visual tokens from both cameras and the state token are concatenated and passed through a 4-layer pre-norm TransformerEncoder:

$$C = \text{Encoder}([v_{\text{table}}; v_{\text{wrist}}; s_{\text{token}}]). \quad (2)$$

The context is $C \in \mathbb{R}^{395 \times 256}$ for v2 and $C \in \mathbb{R}^{513 \times 256}$ for v3. It is computed once per inference chunk and reused across all 10 denoising steps, so the per-step inference cost is dominated by the decoder.

4.5 Action Decoder

The 7-layer decoder is the part that does the work. Each noisy action $x_t^{(i)}$ is projected linearly, added to a learned positional embedding $p_i \in \mathbb{R}^{256}$, and passed through LayerNorm. The flow time $t \in [0, 1]$ is encoded via a sinusoidal embedding with 128 frequencies scaled by 1000, followed by a 2-layer MLP, yielding $\tau \in \mathbb{R}^{256}$. The time embedding is broadcast-added to every action token:

$$\tilde{a}_i = \text{ActionEnc}(x_t^{(i)}) + \tau. \quad (3)$$

Each decoder layer (pre-norm, dropout 0.1) performs self-attention among the 50 action tokens, then cross-attention into the context C as key/value, then a feed-forward block. The strict separation from the encoder is the whole point: action tokens never write back into the visual stream, which was v1’s core failure mode.

4.6 Velocity Head

The decoder output is projected to action space through a 4-operation MLP:

$$v_\theta = W_2 \text{LN}(\text{GELU}(W_1 h)) \in \mathbb{R}^{T \times D}, \quad (4)$$

with $T = 50$ and $D = 6$. All non-backbone parameters are Xavier-initialized.

4.7 Flow Matching

4.7.1 Training Objective

Flow matching [6] learns a velocity field that moves noise ϵ toward the target distribution. For a training chunk $a \in \mathbb{R}^{T \times D}$, time $t \sim \text{Beta}(1.5, 1)$, and noise ϵ , the linear interpolant is

$$x_t = t \cdot \epsilon + (1 - t) \cdot a, \quad (5)$$

so $t = 1$ is pure noise and $t = 0$ is the clean action. The target velocity is $v_t = \epsilon - a$, and the training loss is MSE on the velocity:

$$\mathcal{L} = \mathbb{E}_{t, \epsilon} [\|v_\theta(x_t, C, t) - (\epsilon - a)\|^2]. \quad (6)$$

$\text{Beta}(1.5, 1)$ biases t toward the noisier end of the trajectory (mean 0.6). In practice this gave noticeably cleaner coarse structure in the predicted chunks. Actions crossing episode boundaries are flagged by LeRobot via `action_is_pad`; we re-mask the loss on these positions so the model does not learn to fit padded values.

4.7.2 Inference

At inference we start from $x_1 \sim \mathcal{N}(0, \Sigma_{\text{reg}})$ and run $K = 10$ Euler steps:

$$x_{t-\Delta t} = x_t - \Delta t \cdot v_\theta(x_t, C, t), \quad \Delta t = \frac{1}{10}. \quad (7)$$

Ten steps was the point at which further refinement stopped changing the output qualitatively. Fewer steps gave visibly rougher trajectories; more gave no improvement.

4.8 Correlated Noise Sampler

A standard flow-matching policy uses $\epsilon \sim \mathcal{N}(0, I)$. The difficulty is that at $t \approx 1$, the input is white noise, and the model

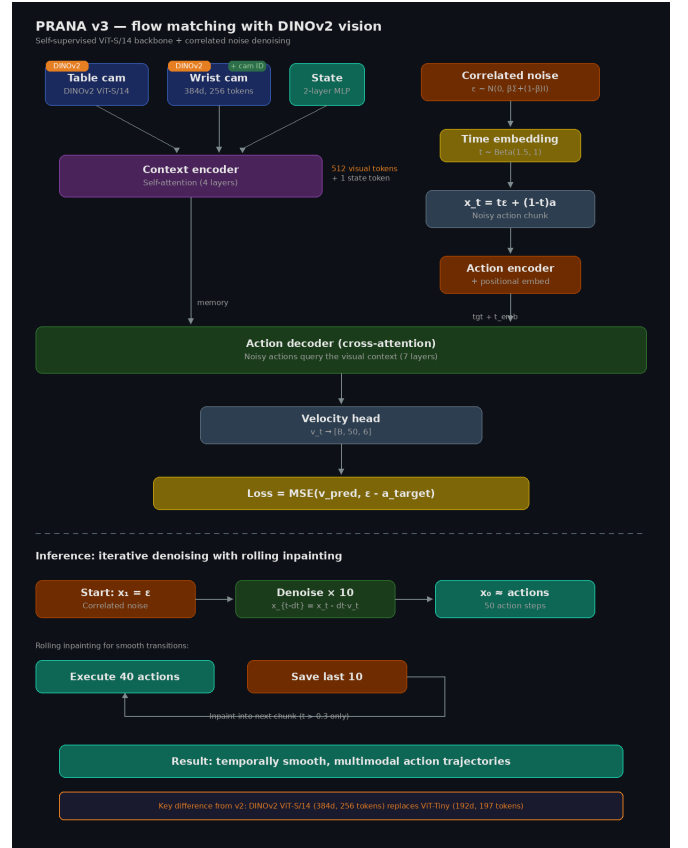


Figure 2: PRANA v3. DINOv2 ViT-S/14 produces 256 spatial tokens per camera at 384 dimensions. Camera-identity embeddings are added at the backbone’s native dimension, then tokens are projected to the 256-dim model space. The 513-token context feeds the same cross-attention decoder as v2.

has to conjure the temporal smoothness and joint coordination of a real trajectory out of it in the first Euler step. In a low-data regime the model does not have the capacity to bridge this gap cleanly, and the result shows up as jitter on hardware.

We replace the inference noise with a distribution whose statistics match real trajectories. The empirical action covariance $\hat{\Sigma} \in \mathbb{R}^{TD \times TD}$ (here 300×300) is computed from centered training chunks:

$$\hat{\Sigma} = \frac{1}{N-1} \sum_{n=1}^N \text{vec}(a^{(n)} - \bar{a}) \text{vec}(a^{(n)} - \bar{a})^\top. \quad (8)$$

To keep the matrix positive-definite under single precision, we apply shrinkage with $\beta = 0.5$:

$$\Sigma_{\text{reg}} = \beta \hat{\Sigma} + (1 - \beta) I, \quad (9)$$

and sample via the Cholesky factor L :

$$\Sigma_{\text{reg}} = LL^\top, \quad \epsilon = Lz, \quad z \sim \mathcal{N}(0, I). \quad (10)$$

The resulting noise has the temporal smoothness and cross-joint coupling of a plausible trajectory, so the first Euler step does not start from an out-of-distribution input.

Workflow. The sampler is decoupled from the main training loop. A standalone `fit_noise.py` script reads the dataset’s

Parquet files, reshapes actions into 50×6 chunks, computes the covariance, applies shrinkage, Cholesky-factors the result, and patches L into the trained checkpoint as a persistent buffer. During the main training run, the sampler is in its default uninitialized state and the model falls back to i.i.d. Gaussian noise; the Cholesky factor is only present at deployment. Empirically, this inference-only swap was the difference between visible oscillation on contact approaches and smooth trajectories on the physical arm. Fitting the sampler before training to get training-time benefits as well would be straightforward, but we did not try it.

4.9 Flow-Consistent Inpainting

Chunked execution introduces discontinuities where two chunks meet. Of the 50 predicted actions in a chunk, we execute the first 40 (n_{action}) and keep the last 10 (n_{inpaint}) as seed actions $\hat{a}_{40:50}$. The initial noise z_{init} is also retained.

During the next chunk’s denoising pass, after the Euler step at time t produces $x_{t-\Delta t}$, the first 10 positions are hard-set to where the seed actions *should* be along the flow trajectory at the new time $t' = t - \Delta t$, but only while $t > 0.3$:

$$x_{\text{target}}^{(1:10)} = t' \cdot z_{\text{init}}^{(1:10)} + (1 - t') \cdot \hat{a}_{40:50}, \quad (11)$$

$$x_{t'}^{(1:10)} \leftarrow m \cdot x_{\text{target}}^{(1:10)} + (1 - m) \cdot x_t^{(1:10)}, \quad (12)$$

with m a binary mask indicating the inpainted positions. The constrained positions stay on the flow manifold rather than being blended toward the target actions directly. With $K = 10$ steps, the constraint is active for the first 7 iterations (while $t \in \{1.0, 0.9, \dots, 0.4\}$) and released for the last 3, so the fine-denoising phase can adapt to the latest observation.

5 Why the Backbone Matters

v3’s single architectural change — swapping ViT-Tiny for DINOv2 ViT-S/14 — produced a clear improvement in deployment quality (Section 8). We find the intuition worth spelling out.

ViT-Tiny is pretrained on ImageNet via supervised classification. Its [CLS] token carries global category information, and its patch tokens tend to encode class-discriminative activations that can fire for semantically similar but spatially distinct regions. For manipulation, what we actually need is the opposite: local spatial structure, clean object boundaries, and a representation in which “the shaft of this screwdriver” and “the rim of this box” occupy different parts of feature space. DINOv2, trained by self-distillation and masked image modeling on a curated 142M-image dataset with no labels, satisfies this better in practice. Its patch tokens are essentially usable as a rough segmentation of the scene.

The larger context (513 vs. 395 tokens) also changes what the decoder’s cross-attention has access to. In v2, each of the 196 spatial tokens per camera covers a 16-pixel region. In v3, each of 256 tokens covers a 14-pixel region at twice the embedding dimensionality before projection. Qualitatively, v3 attention maps at deployment focus more sharply on the screwdriver shaft and the box rim rather than distributing attention over the background. This is consistent with DINOv2’s training objective, but we do not claim a controlled ablation here.

Table 1: Architectural differences across PRANA versions.

Component	v1	v2	v3
Transformer type	Enc. only	Enc-Dec	Enc-Dec
Attention	Self	Cross	Cross
Action prediction	L1 regression	Flow matching	Flow matching
Noise at inference	N/A	Correlated	Correlated
Vision backbone	ViT-Tiny	ViT-Tiny	DINOv2 ViT-S/14
Backbone pretraining	Supervised	Supervised	Self-supervised
Embed dim	192	192	384
Tokens / camera	197 (+CLS)	197 (+CLS)	256 (no CLS)
Context length	395	395	513
Backbone frozen	No	Yes	Yes
Camera ID embed	No	Yes	Yes
Action pos embed	No	Yes	Yes
State encoder	Linear	2-layer MLP	2-layer MLP
Temporal consistency	None	Flow inpaint	Flow inpaint
Enc / dec layers	4 / –	4 / 7	4 / 7
FFN width	1024	512	512
Dropout	0.2	0.1	0.1
Trainable params	~8M	~8M	~8M
Frozen backbone params	–	5.7M	21.9M

DINOv2 adds 21.9M frozen parameters against v2’s 5.7M frozen. Trainable count is unchanged — everything downstream of the projection layer is shared — so training-time memory and gradient-step cost are essentially the same. The extra VRAM cost is about 1.5 GB for the larger backbone activations, which fits comfortably in 8 GB at batch size 8.

6 Architecture Comparison

The “context length” row for v1 counts the vision and state tokens, to match the v2/v3 convention. The additional 16 language tokens and 50 action queries that v1 carries in the same sequence are zero-valued at deployment or learned parameters respectively, and are not part of the observable context.

7 Training Setup

7.1 Dataset

We recorded 123 teleoperation episodes on a SO-ARM101 arm through LeRobot: an initial 83 episodes plus 40 targeted episodes focused on grasp and release. Each episode is a pick-and-place trajectory captured from two cameras (a table-mounted Intel RealSense and a wrist-mounted USB camera) at 30 Hz. The dataset totals 83,966 frames and is hosted on HuggingFace.

7.2 LeRobot Integration

We integrate with LeRobot v0.5.1 via a monkey-patch rather than a fork. At import time, each script overrides `factory.get_policy_class` to return `PranaPolicy` when the policy type is “prana-v2” or “prana-v3”, and patches `factory.make_pre_post_processors` to reuse ACT’s pre/post processors for mean-std normalization of images, states, and actions. This keeps the rest of the LeRobot stack unchanged: dataset loading, chunked action targets via `delta_timestamps`, normalization, AdamW + cosine scheduler, checkpointing, and the deployment entry point are all stock. The policy’s `action_delta_indices` property returns `range(chunk_size)`, which tells LeRobot to load 50-step action chunks directly.

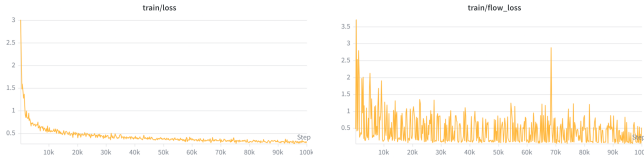


Figure 3: PRANA v2 training over 100K steps. *Left*: total MSE loss, ~ 3.0 to ~ 0.35 . *Right*: flow-velocity MSE.

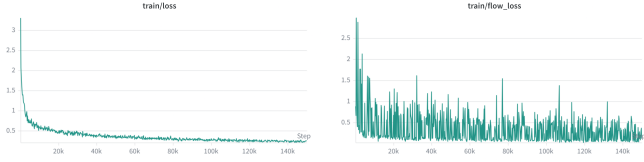


Figure 4: PRANA v3 training over 150K steps. *Left*: total MSE loss, ~ 3.3 to ~ 0.25 . *Right*: flow-velocity MSE.

7.3 Training Configuration

All models train on a single NVIDIA RTX 5060 Laptop GPU (8 GB VRAM, Blackwell sm_120) with PyTorch 2.12.0 nightly on CUDA 12.8. We use AdamW with $\text{lr} = 10^{-4}$, weight decay 10^{-4} , cosine annealing to 10^{-6} , batch size 8, FP16 mixed precision, and gradient clipping at norm 1.0. v2 trained for 100K steps; v3 for 150K steps. Each run took roughly 6–8 hours. When any DINOv2 block is unfrozen, its parameters get a $0.1 \times$ learning rate via a separate optimizer group; in practice we kept the backbone fully frozen.

7.4 Post-Training: Fitting the Noise

After the main training run, `fit_noise.py` is run once. It loads the checkpoint, streams actions from the dataset’s Parquet files, reshapes them into 50×6 chunks, computes the 300×300 covariance, applies shrinkage at $\beta = 0.5$, Cholesky-factors the result, copies the factor into the policy’s `noise_sampler.cholesky_L` buffer, flips the sampler’s `fitted` flag to `True`, and saves the checkpoint back. From that point on the policy samples from $\mathcal{N}(0, \Sigma_{\text{reg}})$ at inference. Skipping this step leaves the sampler at its default — i.i.d. Gaussian — and reproduces the oscillatory behavior that motivated the whole exercise.

8 Results

8.1 Training Dynamics

Figures 3 and 4 show the W&B training curves. Both runs behave similarly in broad strokes: the total MSE loss drops from ~ 3.0 – 3.3 to below 1.0 inside the first 5K steps, as the projection layers and decoder adapt to the flow-matching objective, followed by a long slow descent. v2 ran for 100K steps and ended near 0.35. v3 ran for 150K steps and ended near 0.25; we attribute the improvement to the richer DINOv2 features giving the decoder a cleaner cross-attention target.

The flow-velocity MSE curves (right panels) are noisier than one might expect at first glance. This is not instability; it is the combination of Beta(1.5, 1) time sampling and batch-to-batch variation in t , which makes the loss at any given step depend on which denoising regime happened to be sampled. The periodic sharp spikes line up with episode-boundary padding, which the mask catches but which still causes occasional large-magnitude residuals. v3 had one unusually large spike (~ 2.9) near step

Table 2: Physical deployment on the SO-ARM101 arm.

Metric	v1	v2	v3
Chunk inference	~ 5 ms	~ 22 ms	~ 30 ms
Reaches object	Yes	Yes	Yes
Grasps object	Assisted	Assisted	Yes
Places in box	No	Yes	Yes
Smoothness	Jittery	Smooth	Smother
Position accuracy	Low	Moderate	High

70K that we suspect was a data-loading artifact; the model recovered within about 2K steps, so we did not investigate further. Gradient norms settle to 3–4 after the first few hundred steps, so the clipping threshold of 1.0 is engaged only transiently at the start of training.

8.2 Deployment

The most practically important result is that the correlated-noise sampler is not optional at deployment. Skipping `fit_noise.py` on either v2 or v3 reproduces oscillatory behavior that is visible from across the room. The other practical observation is that camera capture, not policy inference, dominates the control loop: reading both cameras takes roughly 100 ms, which caps the effective control rate at ~ 10 Hz. Policy inference at 22–30 ms per chunk sits well inside that budget, so there is no point optimizing the decoder further without first fixing the capture pipeline.

9 Discussion

9.1 Flow Matching vs. CVAE

ACT’s CVAE needs careful tuning of the KL weight to balance reconstruction and regularization. Flow matching sidesteps this: the noise-to-action map is learned directly, and multimodality emerges from the denoising trajectory rather than from a latent prior. At 123 episodes, having multiple correction opportunities per inference is a real safety net; a single-shot regressor gets exactly one chance.

9.2 Correlated Noise, in One Sentence

Standard flow matching asks the model to climb out of white noise in the first Euler step, which is the hardest step, on the smallest dataset we have. Correlated noise at inference gives it a starting point that is already in the shape of a trajectory.

9.3 Training Workflow and Its Limits

A sharper implementation would fit the sampler before training, so the model sees correlated noise throughout. We instead train with i.i.d. noise and swap in the correlated distribution at inference. The inference benefit was large enough to justify the simpler workflow, but there is almost certainly additional gain available from training-time correlated noise, as in [5].

9.4 Supervised vs. Self-Supervised Vision

The v2→v3 change was deliberately conservative: same decoder, same loss, same noise, different backbone. It produced visibly smoother trajectories and a clear improvement in grasp precision. ViT-Tiny’s features compress class identity; DINOv2’s features preserve spatial structure. Both the richer embeddings and the finer grid matter, but we suspect the pre-training objective is doing most of the work.

9.5 Frozen Backbone as Regularization

At this dataset size, freezing the visual backbone is not a concession — it is the regularizer. Unfrozen ViT-Tiny overfits within a few thousand steps and the feature space collapses. Freezing keeps the pretrained structure intact and pushes the adaptation burden onto the downstream modules, where weight decay and dropout are enough to keep things in line.

9.6 Limitations

We do not condition on force or torque, which limits the quality of contact-rich behaviors. Grasp accuracy is bounded by the limited variation in screwdriver pose in the training set. Ten-step denoising adds latency compared to single-pass methods, though it is not the current bottleneck. The inpainting scheme is a hard-set above a threshold rather than the correlation-aware propagation of [5].

10 Future Work

The obvious extensions are: fitting the noise sampler before training so the model sees correlated noise end-to-end, adding force and torque to the state, implementing full correlation-aware inpainting, conditioning on a task embedding for multi-task operation, scaling the dataset to 400+ episodes to test placement generalization, and a proper ablation over backbones (ConvNeXt, Swin, SigLIP) rather than the single ViT-Tiny→DINOv2 swap we did here.

11 Conclusion

PRANA v2 and v3 show that flow matching, a Cholesky-factored action covariance sampled at inference, and cross-attention between a separate context encoder and action decoder are a workable recipe for manipulation at ~ 120 episodes on a single consumer GPU. Replacing ViT-Tiny with frozen DINOv2 gave further gains without touching the decoder. The

whole pipeline trains in 6–8 hours and runs on a physical SO-ARM101 arm at roughly 10 Hz through LeRobot.

Acknowledgments

We thank the LeRobot team at Hugging Face for maintaining the framework that made this project tractable on a laptop.

References

- [1] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv:2304.13705*, 2023.
- [2] K. Black et al., “ π_0 : A vision-language-action flow model for general robot control,” *arXiv:2410.24164*, 2024.
- [3] Physical Intelligence et al., “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv:2504.16054*, 2025.
- [4] A. Brohan et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv:2307.15818*, 2023.
- [5] I. Larchenko, G. Zarin, and A. Karnatak, “Task adaptation of Vision-Language-Action model: 1st Place Solution for the 2025 BEHAVIOR Challenge,” *arXiv:2512.06951*, 2025.
- [6] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” *arXiv:2210.02747*, 2023.
- [7] C. Chi et al., “Diffusion policy: Visuomotor policy learning via action diffusion,” *IJRR*, 2024.
- [8] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” *ICLR*, 2021.
- [9] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” *arXiv:2506.07339*, 2025.
- [10] M. Oquab et al., “DINOv2: Learning robust visual features without supervision,” *TMLR*, 2024.
- [11] R. Cadene et al., “LeRobot: State-of-the-art Machine Learning for Real-World Robotics in Pytorch,” *GitHub*, 2024.